

CasaRobot: visualización en terminal de `house_world.py`

`common/house_world.py` se puede ejecutar directamente como script: genera una `HouseWorld` (aleatoria, o reproducible con `--seed`) y la dibuja en la terminal con colores ANSI de 24 bits. Es la forma más rápida de enseñar el escenario — por ejemplo, en la presentación de la asignatura — sin escribir ni una línea de código.

```
python common/house_world.py [opciones]
# o, con pixi:
pixi run python common/house_world.py [opciones]
```

Todas las capturas de este documento usan una `--seed` fija para que el resultado sea siempre el mismo; sin `--seed`, cada ejecución genera una casa distinta.

1. Uso básico y reproducibilidad (`--seed`)

Sin argumentos, `house_world.py` genera una casa de 20×14 con 5 habitaciones y una semilla aleatoria distinta en cada ejecución. Fijando `--seed` se obtiene siempre la misma casa, el mismo robot y el mismo listado de habitaciones/objetos — imprescindible para poder enseñar (o depurar) algo reproducible:

```
python common/house_world.py --seed 42
```

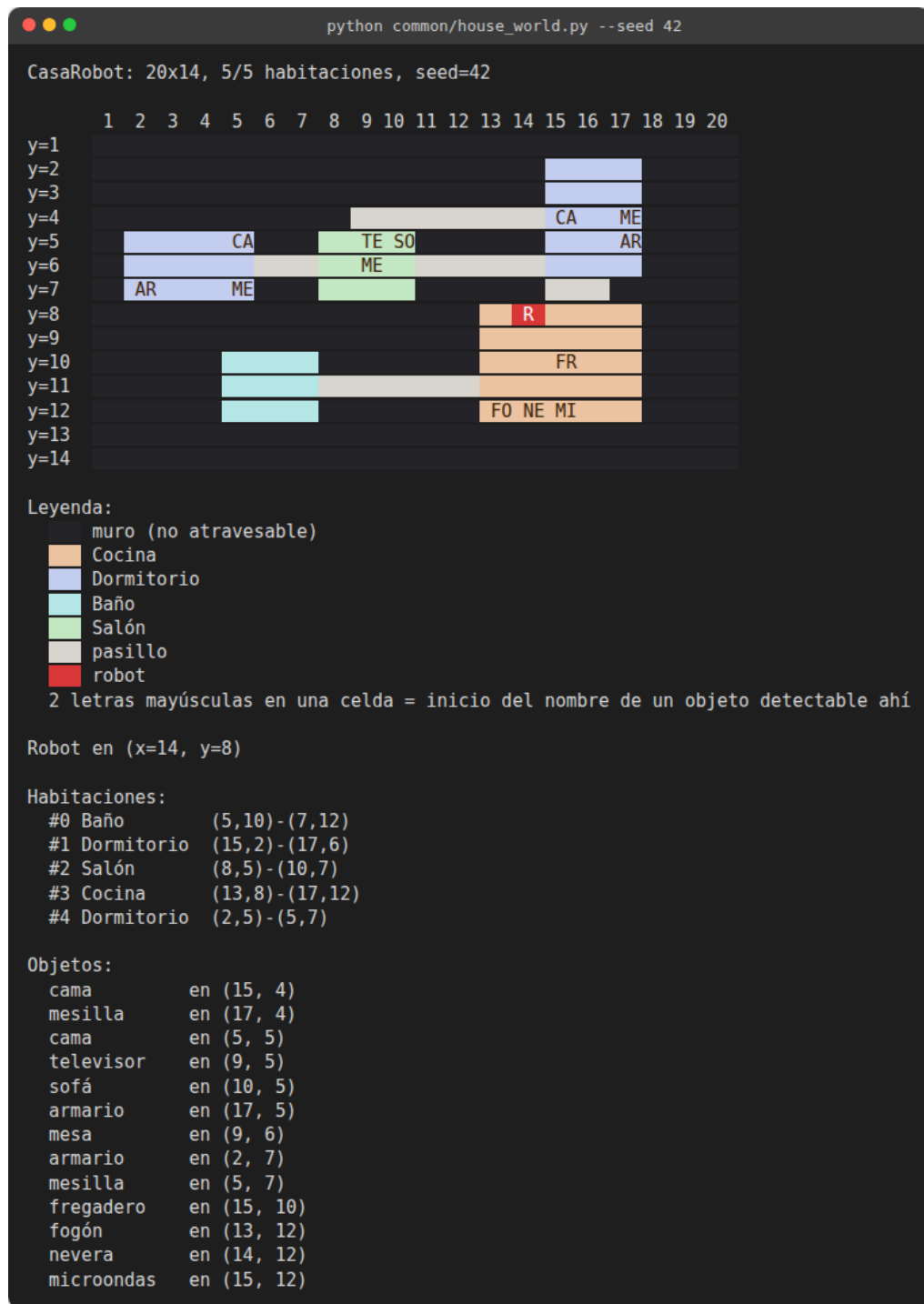


Figura 1: Casa generada con `--seed 42`: mapa coloreado, leyenda, posición del robot, habitaciones y objetos detectables.

2. `--no-color`: alternativa sin ANSI

Si el terminal no soporta color (o se va a volcar la salida a un fichero de texto plano), `--no-color` dibuja el mismo mapa con símbolos ASCII (`#` para muro, `.` para suelo, dos letras mayúsculas para objetos, `[R]` para el robot):

```
python common/house_world.py --seed 42 --no-color
```

```
python common/house_world.py --seed 42 --no-color

CasaRobot: 20x14, 5/5 habitaciones, seed=42

  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
y=1  # # # # # # # # # # # # # # # # # # # #
y=2  # # # # # # # # # # # # # # . . . # # #
y=3  # # # # # # # # # # # # # # . . . # # #
y=4  # # # # # # # # # # . . . . . CA . ME # # #
y=5  # . . . CA # # . TE SO # # # # . . AR # # #
y=6  # . . . . . . ME . . . . . . . # # #
y=7  # AR . . ME # # . . . # # # # . . # # #
y=8  # # # # # # # # # # # # # . R . . . # # #
y=9  # # # # # # # # # # # # # . . . . . # # #
y=10 # # # # . . . # # # # # . . FR . . # # #
y=11 # # # # . . . . . . . . . . . . . # # #
y=12 # # # # . . . # # # # # FO NE MI . . # # #
y=13 # # # # # # # # # # # # # # # # # # # #
y=14 # # # # # # # # # # # # # # # # # # # #

Leyenda:
### muro (no atravesable)
... Cocina
... Dormitorio
... Baño
... Salón
... pasillo
[R] robot
2 letras mayúsculas en una celda = inicio del nombre de un objeto detectable ahí

Robot en (x=14, y=8)

Habitaciones:
#0 Baño      (5,10)-(7,12)
#1 Dormitorio (15,2)-(17,6)
#2 Salón     (8,5)-(10,7)
#3 Cocina    (13,8)-(17,12)
#4 Dormitorio (2,5)-(5,7)

Objetos:
cama        en (15, 4)
mesilla     en (17, 4)
cama        en (5, 5)
televisor   en (9, 5)
sofá        en (10, 5)
armario     en (17, 5)
mesa        en (9, 6)
armario     en (2, 7)
mesilla     en (5, 7)
fregadero   en (15, 10)
fogón       en (13, 12)
nevera      en (14, 12)
microondas  en (15, 12)
```

Figura 2: Misma casa (--seed 42) sin color, con el mapa en ASCII.

3. Tamaño, número de habitaciones y densidad

--width, --height y --rooms controlan el tamaño de la casa y cuántas habitaciones intenta colocar (puede colocar menos si no caben); --min-room/--max-room acotan su tamaño y --density la probabilidad de que aparezca cada objeto típico de cada tipo de habitación:

```
python common/house_world.py --seed 5 --width 30 --height 18 --rooms 8
```

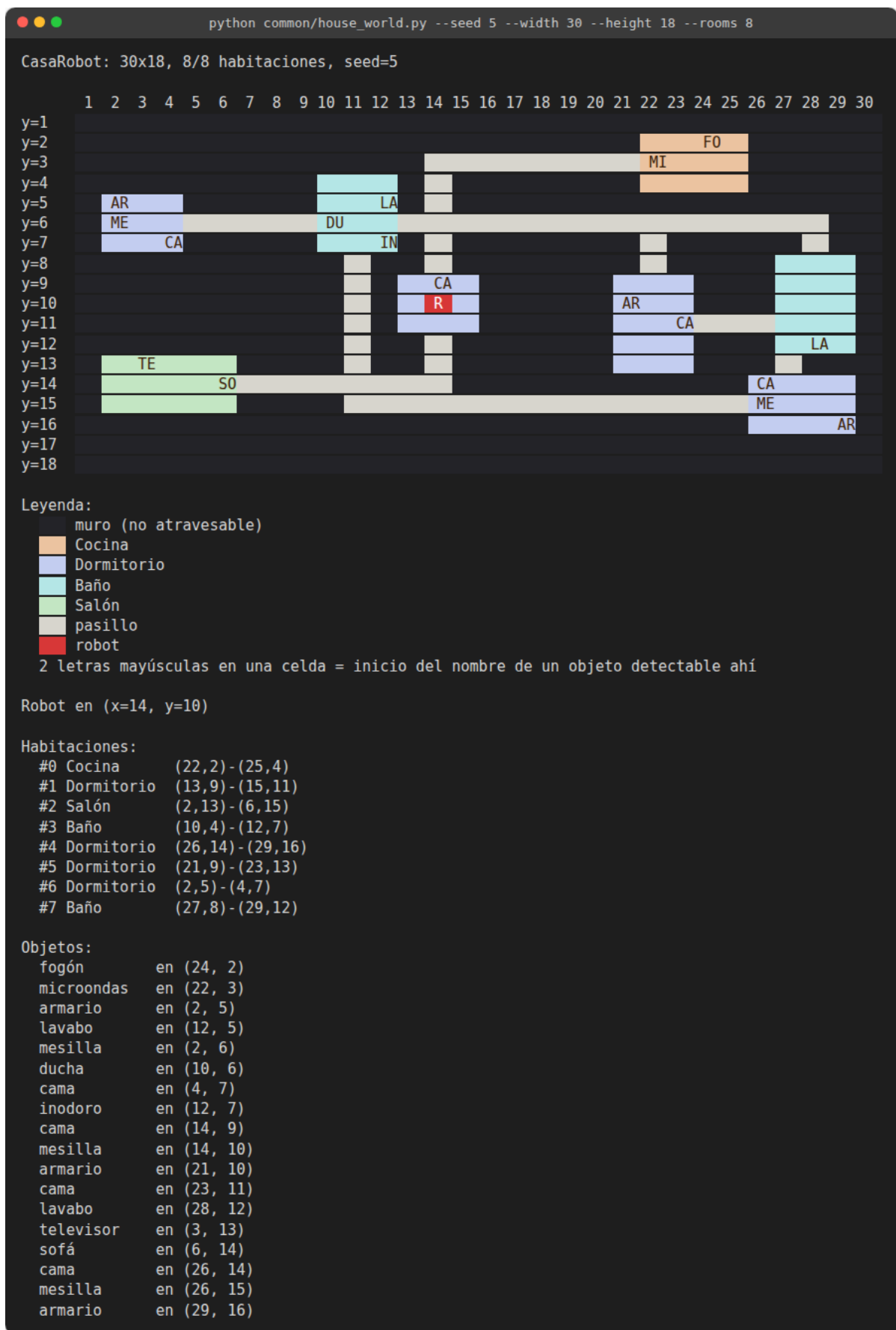


Figura 3: Casa más grande (30 × 18) con 8 habitaciones (--seed 5).

4. `--walk`: paseo aleatorio del robot

`--walk N` simula N pasos aleatorios del robot tras generar la casa, redibujando el mapa completo después de cada paso (incluidas las colisiones contra muros, que no mueven al robot):

```
python common/house_world.py --seed 3 --walk 1
```

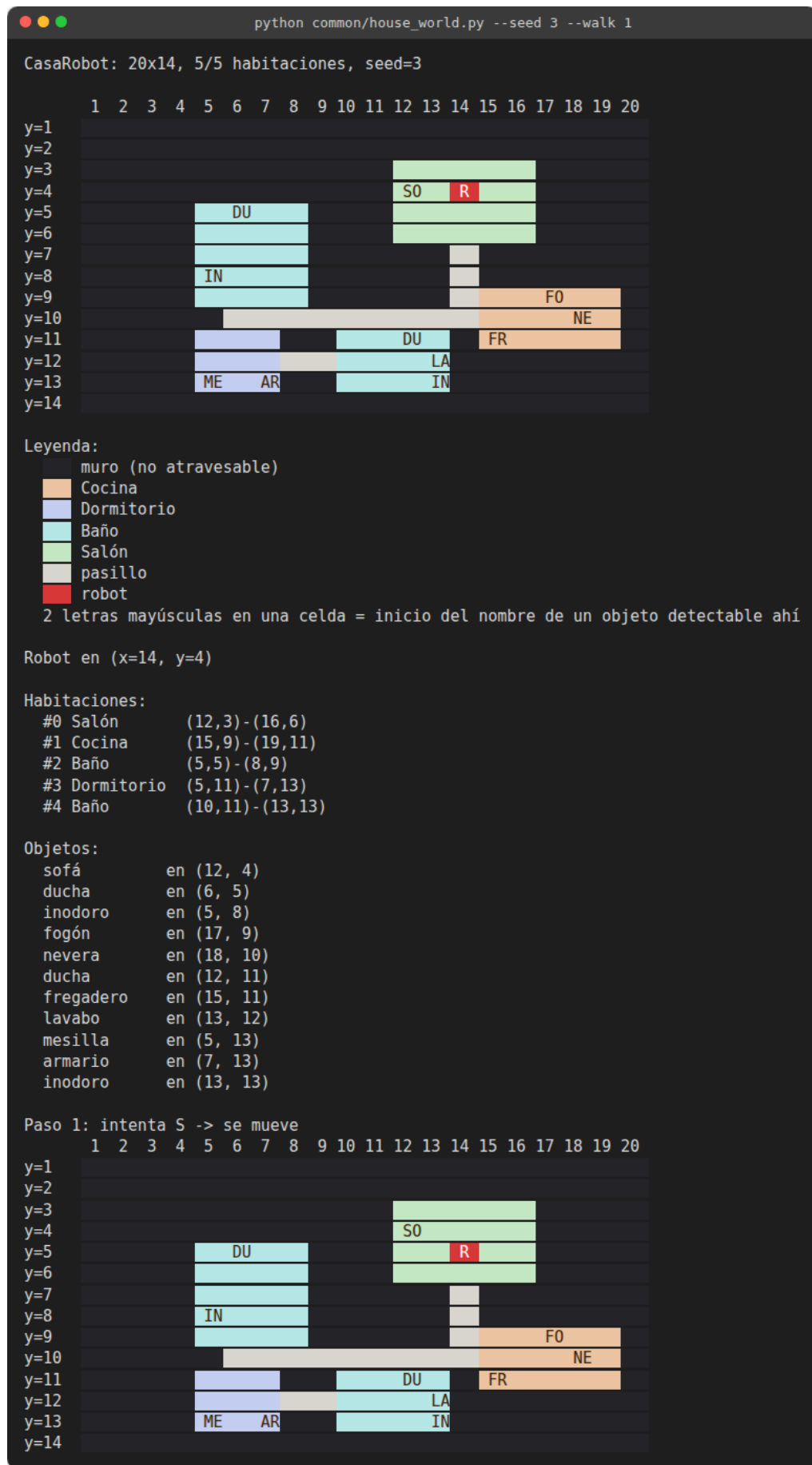


Figura 4: Casa inicial (--seed 3) y el mapa redibujado tras un paso aleatorio del robot con --walk 1.

5. --demo-path: overlay de ruta

--demo-path N superpone al mapa un paseo aleatorio de N pasos a modo de vista previa de cómo se vería el overlay de una ruta (las celdas de la ruta se marcan en morado). **No** es un plan real: la ruta de verdad se calcula en `practical1.py`.

```
python common/house_world.py --seed 7 --demo-path 12
```

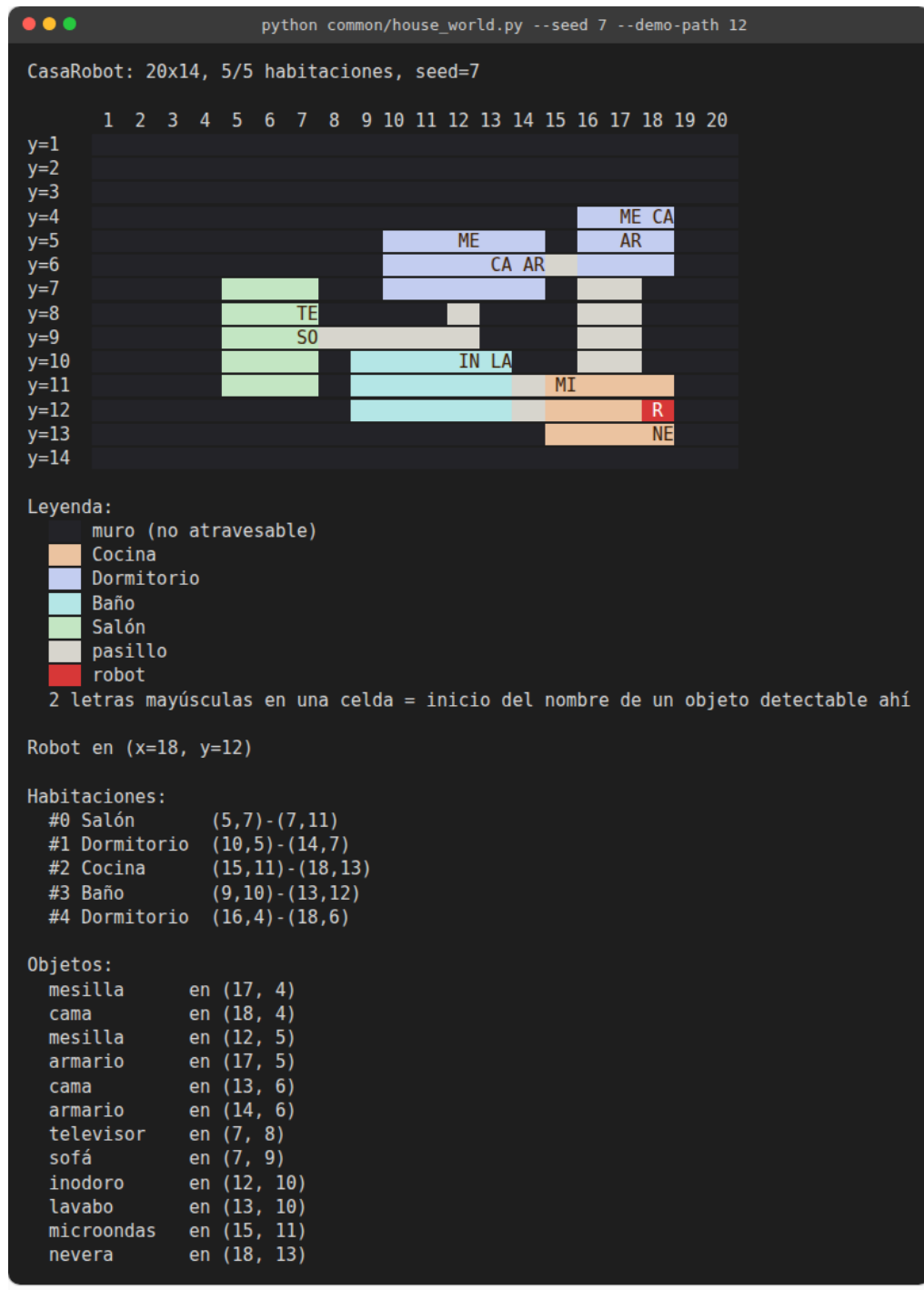


Figura 5: Mapa normal de la casa (--seed 7) que imprime --demo-path antes del overlay.

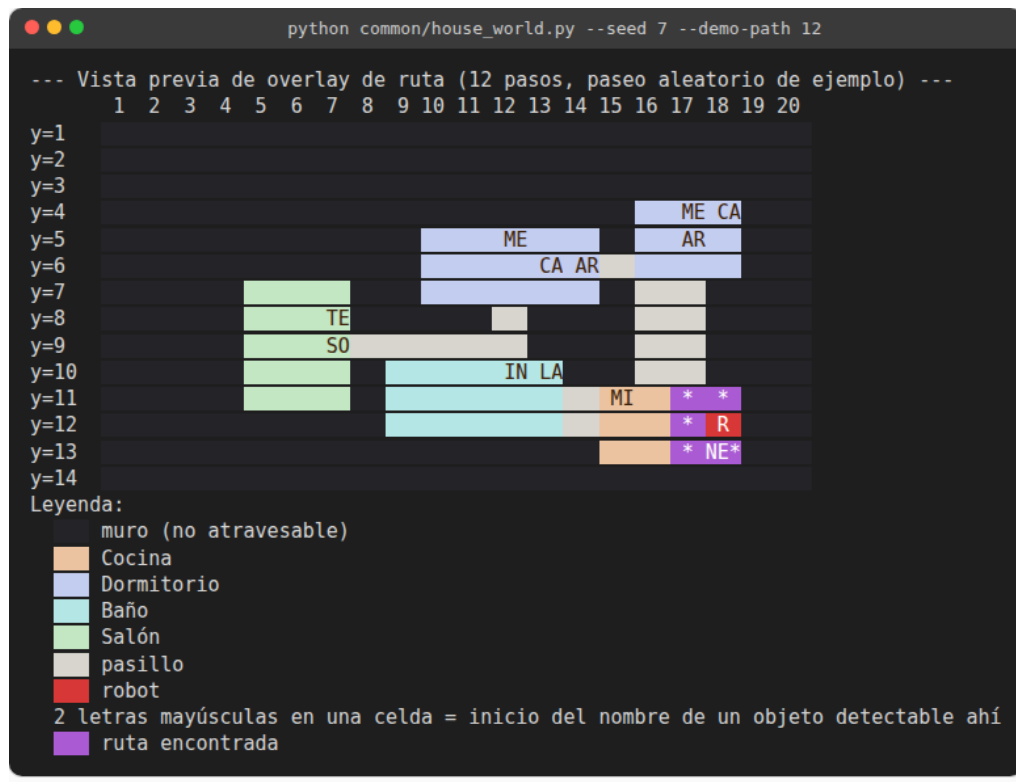


Figura 6: Overlay de ruta de ejemplo a continuación (`--demo-path 12`), sobre la misma casa.

6. `--demo-heat`: overlay de probabilidad

`--demo-heat` superpone una distribución de probabilidad de juguete (rojo = baja probabilidad, verde = alta) a modo de vista previa del overlay de mapa de calor. **No** es un filtro real: la distribución de verdad se calcula en `practica4.py` (HMM).

```
python common/house_world.py --seed 7 --demo-heat
```



Figura 7: Overlay de probabilidad de ejemplo (--seed 7, --demo-heat) bajo el mapa normal de la misma casa.

7. Otras opciones

- --cell-width / --cell-height: tamaño de cada celda en caracteres/filas de terminal, a modo de “zoom” (por defecto 3×1). Útil si la casa no cabe en el ancho del terminal, o para verla más grande en una proyección.
- Todas las opciones se pueden combinar entre sí, por ejemplo --seed junto con --walk, --demo-path o --demo-heat para una demo reproducible paso a paso.
- python common/house_world.py --help lista todas las opciones con su valor por defecto.